MSR Data Challenge: GitSkills Checkpoint 1

Morgan Nicholson, Samuel Bodenheimer, Aidan Sullivan
School of Informatics, Computing and Cyber Systems
CS599: Open Source Software
Dr. Igor Steinmacher
October 22, 2026

I. Introduction

For our research project this semester, we will be investigating the GitSkills dataset from the MSR 2027 Data Mining Challenge. This document will outline our initial progress on our research, methods, and plans for our paper.

II. Dataset Overview

The dataset we will be analyzing is the GitSkills dataset. This dataset comes from a paper created by Giuseppe Destefanis, Daniel Graziotin, Matteo Vaccargiu and Marco Ortu. Their paper proposes a dataset representing logs from 3,797,117 `SKILL.md` files from 282,200 repositories on GitHub [1]. The data was collected from GitHub throughout July 2026. Artifacts in this dataset represent a single skill file, as well as associated metadata about its repository and referenced files. A link to the paper may be found in the references section, or you may access it directly via the following link: https://arxiv.org/abs/2608.13153.

This paper will be referencing version 3 of the dataset published on September 11, 2026, the parquet file for which is available at their hugging face repository[1]. Use of this dataset is governed by the Creative Commons Attribution 4.0 International License , which allows for sharing and adaptation of the dataset as long as proper attribution is given.

III. Data Structure

This dataset contains 5 tables of data: artifacts, repositories, artifact siblings, history, and mining run data. The latter of which is not relevant to studying the dataset, so we will focus only on the first 4 tables. In the artifact table, each row corresponds to a single `SKILL.md` file. Repository metadata is included for referencing, and contains the repository name, owner, and license. Artifact siblings are files that are referenced within the skills file and stored in the same directory as the skills file. And finally, history data contains information on the commit history of the skills file, including the first and last commit dates, and the author type (user or bot). Our study will working mainly with the artifact table and its associated fields. Each artifact contains the following fields (not including flags):

- *Repository Name*: Name of the repository containing the `SKILL.md` file.
- *Artifact Path*: Path to the `SKILL.md` file within the repository.
- *Entry Name*: Assigned name of the skill
- *Entry Type*: Type of the skill
- *Entry Size*: Size in bytes of the body of the skill
- *Entry SHA*: SHA hash of the entry (Identical entries will have the same SHA)
- *Content*: Text in the body of the skill

The paper mentions that some entries have identical SHA values, which indicates that there is a duplicate artifact in the dataset [1]. This is something we will consider when filtering into our working dataset. This study will mainly be extracting data from the *content* field for each artifact, as we will initially be gathering the names of the tools mentioned in each skill.

IV. Research Questions and Related Work

In this paper, we use *tool* to mean an external dependency or resource that a skills file names or references in instructions for completing a task. This is an important definition for our study, it includes software packages, developer tools, APIs, platforms, and services, but does not imply that the referenced resource is necessarily callable by the agent. For example, `npm` or a named package may be referenced in a React skill, while `Elasticsearch` may be referenced in an e-commerce skill. These cases are referred to as tools even if the file does not provide a direct integration. We also distinguish between tools and tool projects, multiple tools may resolve to the same project, and a single service or product may have several names or components.

RQ1: What are the most common tools mentioned in skills files?

Our first goal is to compile a list of tools used by agent skills. Identifying frequently referenced tools can reveal which parts of the software ecosystem skills are designed to help users navigate, and can provide a basis for studying dependencies and relationships between skills. For OSS maintainers, such patterns may also indicate where skills could influence discovery and use of projects. This research will help us understand what kinds of agent services skills files aim to direct. Additionally, once we have a list of common tools it will be possible to perform additional analysis of the relationships between skills files and their tools.

Prior research provides a broad picture of skill content and function, including the concentration of skills in software-

[1] https://huggingface.co/datasets/mvaccargiu/gitskills

engineering tasks [2]. What remains less clear for our purposes is the identity and frequency of the external resources named within skill instructions.

RQ2: What is the license distribution of the tools mentioned by skills files?

This question matters because the license of a referenced tool can affect whether and how developers may use, modify, redistribute, or incorporate it into software projects. Understanding the distribution can help characterize whether skills predominantly direct users toward openly licensed software, proprietary products or services, or a mixture of these. This is relevant to engineering decisions involving dependency selection, compliance review, and reproducibility. A mention in a skills file, however, may not indicate that the skill's author or user has reviewed or complied with the tool's license.

Related work on agent tool use addresses questions such as how tools are exposed and secured in agent systems [3]. That work motivates attention to the external systems used by agents, but does not answer the question posed here.

RQ3: What is the license distribution of the projects that host tools mentioned in skills files, grouped by the freedom each license provides?

RQ3 shifts the unit of analysis from the tool as named in a skills file to the project that hosts or provides it. The distinction is important because a tool name may refer to a product with multiple components, and a tool may be distributed by a project whose repository license differs from the terms governing a hosted service. This question matters to OSS and software engineering because license names alone do not fully describe the permissions and obligations relevant to reuse. Grouping licenses by the freedoms they provide can make the distribution more interpretable in terms of what users are permitted to do with the project.

Prior work has examined developers' perspectives on licensing and copyright issues related to generative AI for software development, using a survey of 574 developers and follow-up interviews [4]. That work informs the importance of licensing and ownership questions in contemporary development, but it measures developer perspectives rather than the license terms of projects referenced by agent skills. Another study likewise provides context on the skills ecosystem, but does not, as framed here, provide the project-level, rights-oriented license distribution we seek [2]. Our study adds a project-level view linking skill references to hosting projects and classifying identified licenses by the permissions they grant.

V. Proposed Method

We use an AI-assisted mixed-methods design implemented in Python with Polars DataFrames to analyze GitSkills [1], downloaded from Hugging Face,[2] and we fix a seed for stochastic steps (sampling and applicable model calls) so the run is reproducible. Fig. 1 summarizes the five major stages, while the appendix gives the extended flowchart (accuracy checks, manual gap fill, and merges).

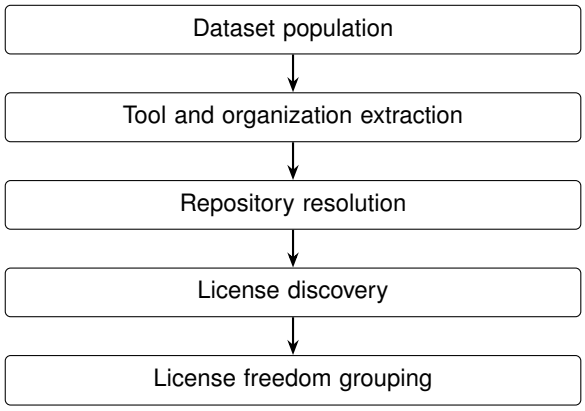


Fig. 1. High-level analysis pipeline. See the appendix for the extended diagram (accuracy checks, manual gap fill, and merges).

For each AI stage we apply a quality gate on a random sample of outputs (sample size TBD), and tie expected accuracy to the operating threshold (e.g., a GLiNER [5] score threshold of 0.8 requires at least 80% sample accuracy). If the sample is below target, we adjust the threshold and re-run or re-filter that stage, and if it meets the target we manually resolve gaps, merge automatic and manual results (dropping non-existing entities), and only then advance.

A. Dataset population

We download GitSkills and load the Parquet tables with Polars, while population filters and analysis slices remain TBD (candidates include `dedup_primary`, `frontmatter_valid`, exact `SKILL.md` filename, and `location_class`). For extraction we use only the `SKILL.md` content and never sibling files, keeping a skill when that content is textual and skipping non-text cases such as CSS.

B. Tool and organization extraction

We extract taught *tools* and *organizations* with GLiNER (initial score threshold 0.8), where a tool is a product the skill teaches the agent to use, not the host skill directory, publishing repository, or an in-text attribution link. Organizations support later repository search, while RQ1 frequencies count taught tools. After the quality gate, empty or unresolved rows are filled manually and merged before repository resolution.

C. Repository resolution

We resolve tools sequentially, reusing a project when a tool is already linked and otherwise querying versioning platforms (e.g., GitHub or GitLab) and using a reasoning model (TBD) to select the owning repository from API candidates, with open-web search as a fallback. The same accuracy gate, manual gap fill, and merge close this stage before license discovery.

[2]https://huggingface.co/datasets/mvaccargiu/gitskills

D. License discovery

For each resolved repository we scan the repository root for conventional license filenames (e.g., `LICENSE`, `COPYING`), extract the license identity, and link it to the tool, with gaps again going through sample validation (when AI-assisted), manual review, and merge.

E. License freedom grouping

Confirmed licenses are grouped by *freedom*, meaning what the license allows or denies doing with the tool (not only SPDX[3] string frequency), and these groups support RQ2 (license distribution of tools) and RQ3 (freedom grouping of projects that host tools mentioned in skills).

Units of analysis follow the pipeline, so RQ1 uses distinct skills and tool mentions, RQ2 uses confirmed tools with licenses, and RQ3 uses resolved projects with a freedom group. Limitations include TBD sample sizes and population slices, a TBD repository ranking model, wrong-repository selection, root-only license scans, and residual API non-determinism where seeds are not honored.

VI. Data Quality

Our quality concern is the AI-assisted steps in the pipeline and how we keep their outputs trustworthy before later analysis uses them. We apply AI for taught-tool and organization extraction with GLiNER [5], for selecting an owning repository with a reasoning model (TBD) when a tool is not already linked, and for license-identity gaps when that step is AI-assisted. Extraction reads only textual skill-file content from GitSkills [1], never sibling files.

For each AI stage we draw a random sample of outputs (sample size TBD) and score accuracy against the operating threshold. For example, a GLiNER score threshold of 0.8 requires at least 80% sample accuracy. If the sample falls short, we adjust the threshold and re-run or re-filter that stage. If it meets the target, we manually resolve remaining gaps, merge automatic and manual results while dropping non-existing entities, and only then advance. The appendix flowchart shows this gate, gap fill, and merge after each major AI stage.

Without AI assistance the same steps would be full manual extraction, and checking every AI output by hand would take effort on the same order as that work, but either path would limit how many skills, tools, and repositories we can cover. We therefore accept a bounded residual error under the sample accuracy gate, in the same spirit as a population survey that estimates from a sample instead of visiting every case, so the study can scale. Stages that miss the bar are retuned or re-filtered, and only after the bar is met do we manually fill remaining gaps and merge before advancing.

The gate is meant to catch common failure modes of these models. On extraction, that includes treating a host skill directory, a publishing repository, or an in-text attribution link as a taught tool. On repository resolution, it includes choosing the wrong project among API or web-search candidates. We also fix a seed for sampling and applicable model calls, while noting residual API non-determinism where a seed is not honored. Open parameters remain the sample sizes and the concrete ranking model, both still TBD, so we do not claim measured accuracies in this checkpoint.

[3]Software Package Data Exchange (SPDX) provides short standard license identifiers such as MIT and Apache-2.0. See https://spdx.org/licenses/.

VII. Plan

All of the team members will share responsibility for study design, implementation, validation, analysis, and writing. Each member will contribute substantially and review the full pipeline so that all understand the design and its limitations.

A. Responsibilities

The project responsibilities are dataset profiling, population selection, tool extraction, license identification and freedom grouping, quality checks, analysis of RQ1–RQ3, and preparation of the paper. The team will document decisions, code, and validation results in the shared repository.

B. Milestones

- **Before November 3:** Profile GitSkills, decide on filters, prototype tool extraction, and draft license grouping rules.
- **Before December 3:** Complete tool extraction, license discovery, validation, and the RQ1–RQ3 analyses.
- **Before December 9:** Incorporate feedback, rerun and check the analysis, finalize figures and results, and submit the paper.

C. Risks and contingencies

Repository discovery depends on confirmed tool mentions, and license analysis depends on discovered repositories. This largest risk for this project is that we will not be able to get satisfactory results from the tool, repository, and license discovery methodology. In the case of this happening, we will move to a less automated strategy utilizing manual analyses and tools such as the GitHub repository search API. If quality checks fail, the team will revise the relevant settings, rerun the stage, and review remaining gaps manually. If APIs, repository candidates, or license files are unavailable or ambiguous, unresolved cases will be recorded rather than guessed. If needed, the team will report a clearly defined subset and explain its effect on the research questions.

Appendix A
Appendix

Fig. 2 expands Fig. 1 from the Proposed Method. Each major stage keeps a top-to-bottom order. An automated step is followed by a sample accuracy check, then threshold adjustment or manual review of gaps, then a merge before the next stage.

REFERENCES

[1] G. Destefanis, D. Graziotin, M. Vaccargiu, and M. Ortu, "GitSkills: A dataset of agent skills on GitHub," in *Proceedings of the 24th IEEE/ACM International Conference on Mining Software Repositories*, ser. MSR '27. Piscataway, NJ, USA: IEEE, 2027, p. To Appear. [Online]. Available: https://arxiv.org/abs/2608.10906

[2] H. Xing, H. Zhuang, X. Zhao, Y. Huang, Z. Tang, and X. Zhang, "Recipes for agents: Understanding skills and their open questions," in *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*, ser. KDD '26. New York, NY, USA: Association for Computing Machinery, 2026, p. 13245–13251. [Online]. Available: https://doi.org/10.1145/3770855.3818652

[3] F. J. Arceo and V. P. Narsing, "Securing the agent: Vendor-neutral, multitenant enterprise retrieval and tool use," in *Proceedings of the ACM Conference on AI and Agentic Systems*, ser. CAIS '26. New York, NY, USA: Association for Computing Machinery, 2026, p. 862–872. [Online]. Available: https://doi.org/10.1145/3786335.3813145

[4] T. Stalnaker, N. Wintersgill, O. Chaparro, L. A. Heymann, M. D. Penta, D. M. German, and D. Poshyvanyk, "Developer perspectives on licensing and copyright issues arising from generative ai for software development," *ACM Trans. Softw. Eng. Methodol.*, vol. 35, no. 2, Jan. 2026. [Online]. Available: https://doi.org/10.1145/3743133

[5] U. Zaratiana, N. Tomeh, P. Holat, and T. Charnois, "GLiNER: Generalist model for named entity recognition using bidirectional transformer," 2023. [Online]. Available: https://arxiv.org/abs/2311.08526

[6] W. Xu, K. Gao, H. He, and M. Zhou, "Licoeval: Evaluating llms on license compliance in code generation," in *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering*, ser. ICSE '25. IEEE Press, 2025, p. 1665–1677. [Online]. Available: https://doi.org/10.1109/ICSE55347.2025.00052

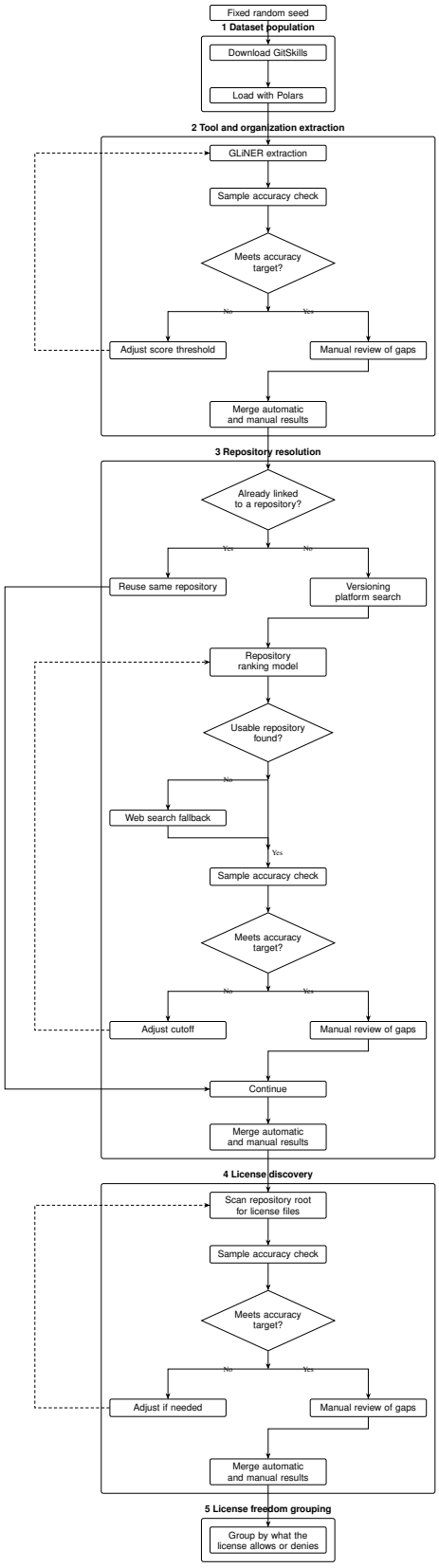


Fig. 2. Extended analysis pipeline. Decision exits leave from the bottom of each diamond (Yes/No). Dashed arrows are threshold-adjustment retries. The repository-ranking model is TBD. Population slices and accuracy sample sizes are TBD.